

Chapter 3: Large Language Models (LLMs)

Learning Objectives

By the end of this chapter, you will:

- Understand how Large Language Models (LLMs) work and the key concepts behind them.
- Differentiate between tokens, embeddings, context windows, and fine-tuning.
- Learn how AI TPMs evaluate, select, and manage LLM implementations in enterprise environments.

Beginner Explanation

Large Language Models (LLMs) are AI systems trained on massive amounts of text data to understand and generate human-like language.

Examples of popular LLMs include:

- GPT-5.5
- Claude
- Gemini
- Llama
- Mistral
- DeepSeek

Think of an LLM as a highly capable prediction engine.

When you type:

"Write an email to my manager requesting leave."

The model predicts the most likely sequence of words based on patterns it learned during training.

Key Concepts

Term	Simple Explanation
Token	Small pieces of text processed by an LLM
Context Window	Amount of information an LLM can remember in one conversation
Embeddings	Numerical representation of text
Fine-Tuning	Training a model for specific tasks
Inference	Generating responses from the model

Example

Sentence:

"Azure OpenAI helps enterprises."

May become:

- Azure
- Open
- AI
- helps
- enterprises

These pieces are called tokens.

Intermediate Explanation

Most enterprises do not build their own LLMs because:

- Training costs are extremely high.
- Infrastructure requirements are significant.
- Continuous maintenance is required.
- Specialized expertise is needed.

Instead, organizations typically:

1. Consume managed models.
2. Build Retrieval-Augmented Generation (RAG) systems.
3. Fine-tune when necessary.
4. Implement governance layers.

Enterprise Decision Framework

Factor	Consideration
Cost	Token consumption
Performance	Response quality
Security	Data residency
Scalability	User volume
Compliance	Industry regulations
Latency	Response time

Common Enterprise Patterns

Pattern 1: Direct LLM Access

User → Application → LLM

Pattern 2: RAG

User → Search → LLM → Response

Pattern 3: Agentic Workflow

User → Agent → Tools → LLM → Response

Why This Matters for an AI TPM

Responsibility	Impact
Delivery	Ensures the appropriate LLM is selected for business requirements.
Risk	Addresses hallucinations, compliance, and privacy concerns.
Stakeholder Management	Aligns technical teams and business leaders on model capabilities.
Budget	Tracks token usage, API consumption, and infrastructure costs.

Enterprise Example

Business Problem

A healthcare organization wanted to provide employees with a virtual assistant capable of answering questions related to HR policies, insurance, and onboarding procedures.

Solution

Deploy an enterprise knowledge assistant using Azure OpenAI and Azure AI Search.

Architecture

- Azure OpenAI
- Azure AI Search
- Azure Blob Storage
- Azure Functions
- Microsoft Teams Integration

Outcome

Metric	Result
Employee Adoption	87%
HR Ticket Reduction	52%
Average Response Time	<4 Seconds
Annual Savings	~\$300,000

Lessons Learned

- LLMs should not operate without enterprise context.
- RAG significantly improves answer quality.
- Cost monitoring is essential.
- Human validation remains necessary for sensitive topics.

Azure Implementation

Services Used

Service	Purpose
Azure OpenAI	LLM hosting and inference
Azure AI Search	Knowledge retrieval
Azure Functions	API orchestration
Azure AI Foundry	Model management and evaluations

Implementation Workflow

Step 1: Select a Model

Example considerations:

Requirement	Model Characteristic
Low Latency	Smaller models
High Accuracy	Larger models
Cost Efficiency	Balanced models
Complex Reasoning	Advanced reasoning models

Step 2: Configure Security

Implement:

- Role-Based Access Control (RBAC)
 - Private Networking
 - Encryption
 - Audit Logging
-

Step 3: Build Knowledge Integration

- Upload enterprise documents.
 - Create embeddings.
 - Configure search indexes.
 - Enable semantic retrieval.
-

Step 4: Monitor Performance

Track:

- Tokens per request
 - Latency
 - User satisfaction
 - Error rates
 - Monthly spend
-

Mermaid Diagram

```
graph TD
  A[Enterprise User]
  A --> B[Application Layer]
  B --> C[Azure OpenAI]
  C --> D[Azure AI Search]
  D --> E[Enterprise Documents]
  E --> F[Generated Response]
  F --> G[User]
```

TPM Checklist

- ☐ Define business objectives.
 - ☐ Select an appropriate LLM.
 - ☐ Estimate monthly token costs.
 - ☐ Define performance metrics.
 - ☐ Conduct security assessments.
 - ☐ Establish governance controls.
 - ☐ Configure monitoring.
 - ☐ Define escalation procedures.
 - ☐ Perform user acceptance testing.
 - ☐ Obtain executive sign-off.
-

Templates

LLM Selection Matrix

Criteria	Weight	Score
Cost	20%	
Performance	25%	
Security	20%	
Latency	15%	
Scalability	10%	
Supportability	10%	

Token Cost Tracker

Metric	Monthly Value
Total Requests	
Average Tokens	
Total Tokens	
Estimated Cost	
Budget Variance	

Risk Assessment

Risk	Severity	Mitigation
Hallucinations	High	Human review

Risk	Severity	Mitigation
Data Leakage	High	RBAC
Downtime	Medium	Multi-region deployment
Cost Overruns	Medium	Budget alerts

Common Mistakes

1. Selecting models solely based on popularity.
2. Ignoring token consumption costs.
3. Assuming larger models are always better.
4. Skipping governance requirements.
5. Failing to integrate enterprise knowledge.
6. Underestimating latency requirements.
7. Neglecting monitoring and observability.

Key Takeaways

- Large Language Models are prediction engines trained on vast datasets.
- Most enterprises consume managed LLMs rather than building their own.
- Understanding tokens, embeddings, and context windows is critical for AI TPMs.
- Successful implementations require governance, monitoring, and cost management.
- Azure OpenAI combined with Azure AI Search provides a scalable enterprise architecture.
- AI TPMs must balance performance, risk, cost, and business value when selecting LLM solutions.

Footer